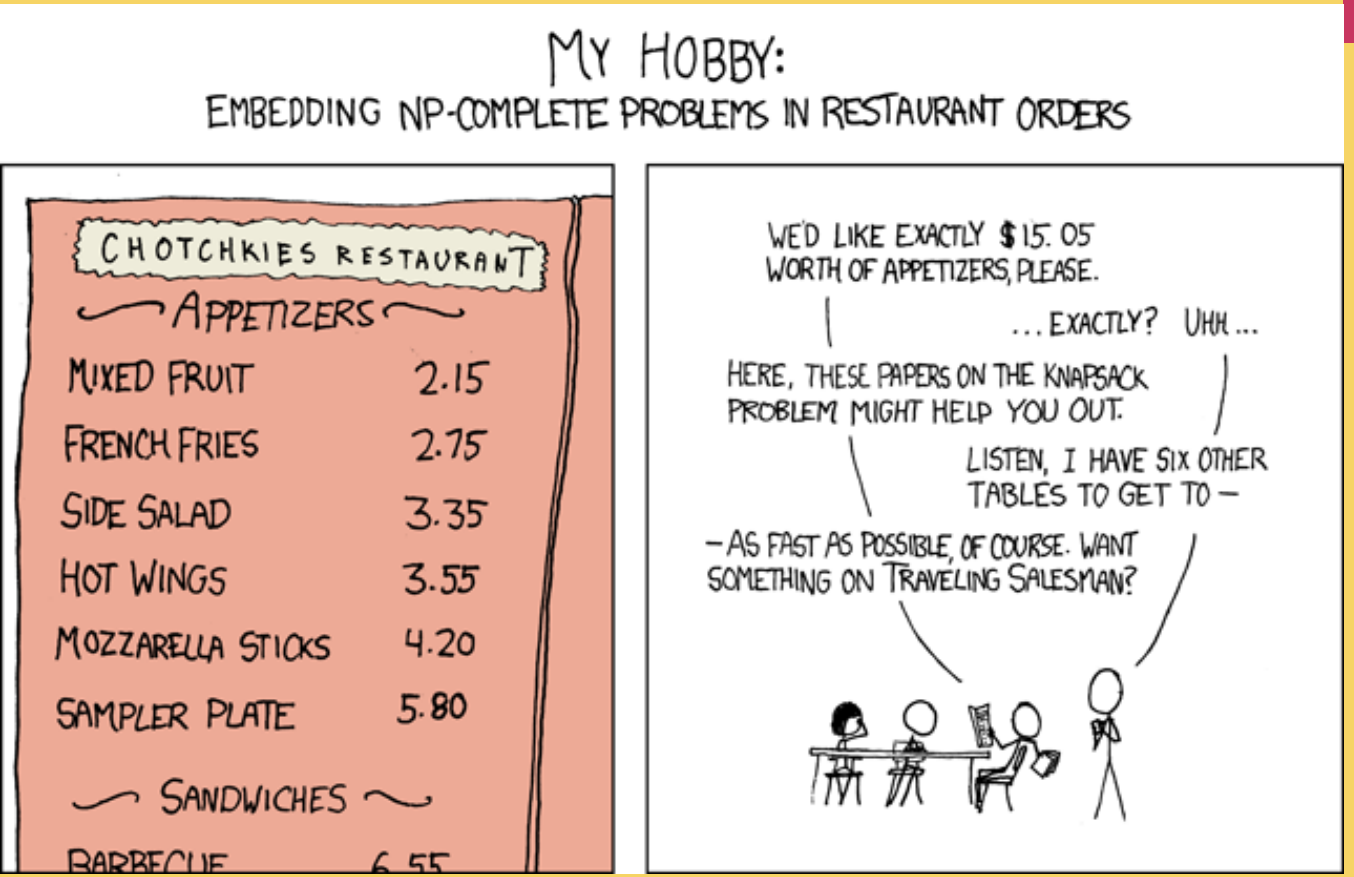
HW 8 due this Friday.
HW 9 coming soon.

No Quiz this week.



https://xkcd.com/287/

Class 25:
Class NP

University of Virginia
CS3120: DMT2
https://weikailin.github.io/cs3120-toc
Wei-Kai Lin

Plan

Class NP

Definition of **NP**

NP-*Hardness*

NP-*Completeness*

Textbook [TCS] Section 15

https://introtcs.org/public/lec_13_Cook_Levin.html
NP, NP Completeness

Gemini generated cover image.

Recap

Karp Reduction

Definition 14.1 (Polynomial-time reductions)

Let $F, G : \{0,1\}^* \to \{0,1\}$. We say that F *reduces to* G, denoted by $F \leq_p G$ if there is a polynomial-time computable $R : \{0,1\}^* \to \{0,1\}^*$ such that for every $x \in \{0,1\}^*$,

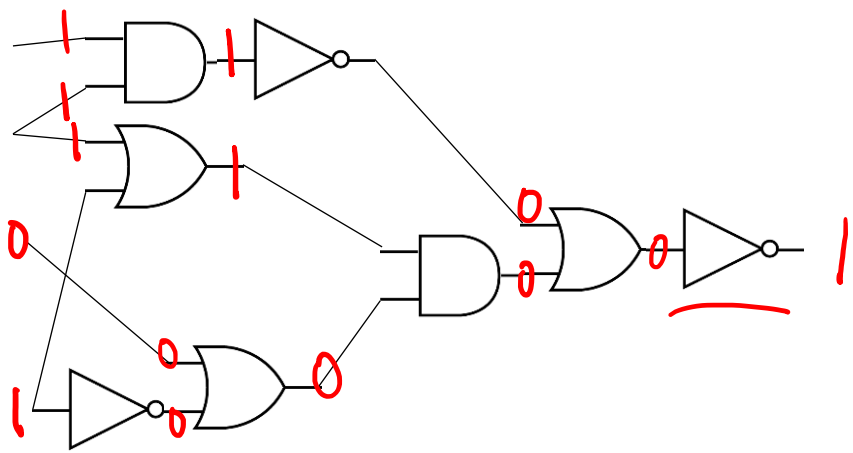
$$F(x) = G(R(x)) \ . \quad (14.1)$$

We say that F and G have *equivalent complexity* if $F \leq_p G$ and $G \leq_p F$.

Note: This is the “Karp reduction” definition. (Cook reduction is more general)

Recap: Satisfiability Problems

CircuitSAT :
$\{C : C$ is a Boolean circuit such that $\exists x,\ C(x) = 1\}$



3-SAT

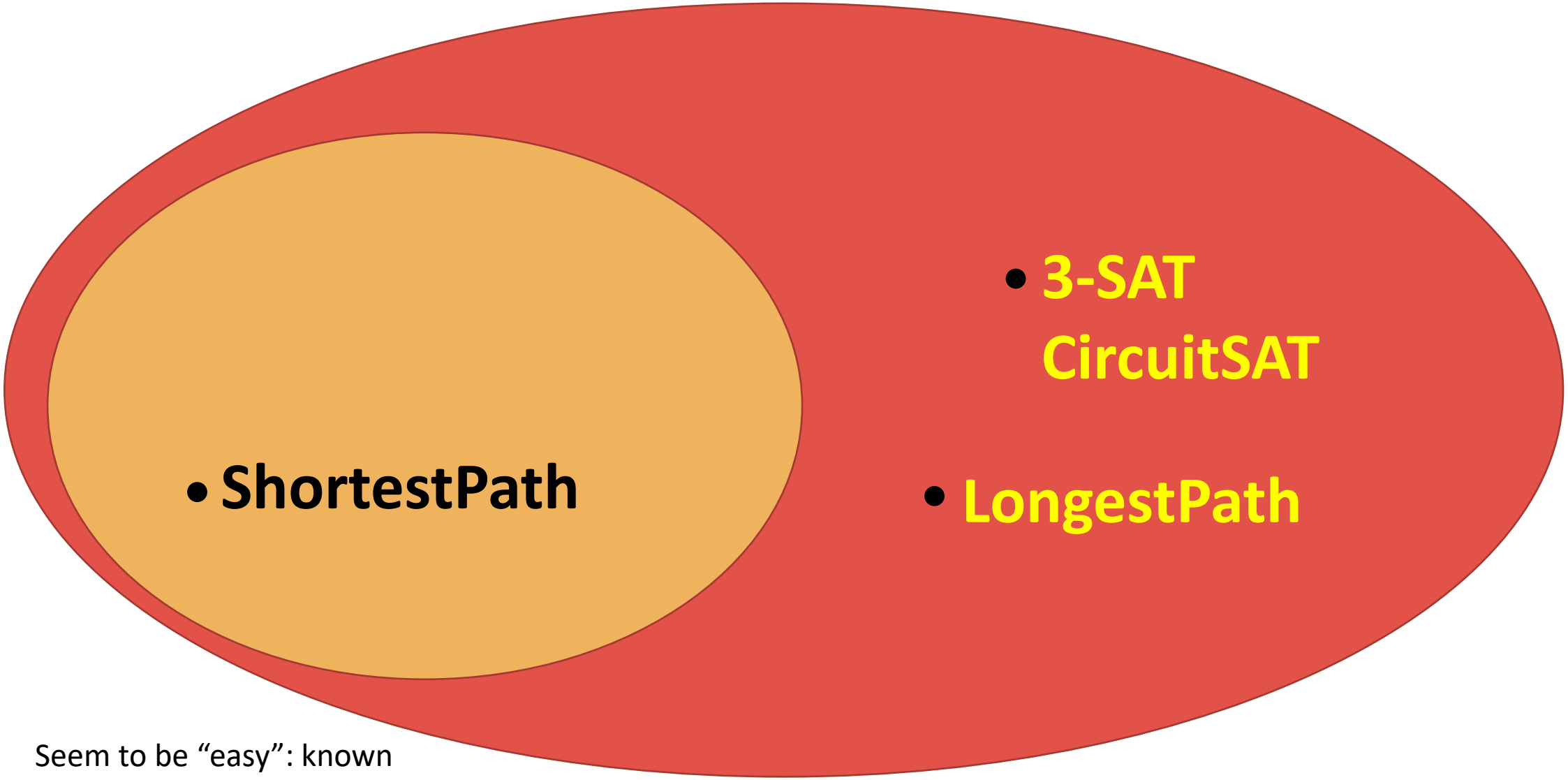
Given a 3-CNF formula F, is there an assignment of true/false such that F(assignment)=1?

$$F(x_1, x_2, x_3, x_4) = (x_1 \vee x_2 \vee x_3) \wedge (x_1 \vee \overline{x_2} \vee x_2) \wedge (x_4 \vee x_2 \vee \overline{x_3}) \wedge (x_3 \vee \overline{x_1} \vee x_4) \wedge (\overline{x_1} \vee \overline{x_2} \vee \overline{x_3})$$

$x_1 = true$
$x_2 = false$
$x_3 = false$
$x_4 = true$

Definition:
3-SAT(F) = 1 if there exists x such that F(x) = 1
0 otherwise.

6



Seem to be "easy": known polynomial-time algorithms

Class **P** = $\bigcup_{c \in \{1,2,3,\ldots\}} TIME_{TM}(n^c)$

Class **EXP** = $\bigcup_{c \in \{1,2,3,\ldots\}} TIME_{TM}(2^{n^c})$

Complexity Class NP

NP stands for “non-deterministic” polynomial time.

Revisiting the hard problems we have seen

What do these problems have in common?
CircuitSAT, LongestPath, 3-SAT

- We do not know any poly-time algorithm for them.
- In class **EXP**
- There are some poly-time reductions between them

Can verify solns of problem instances quick!

YES

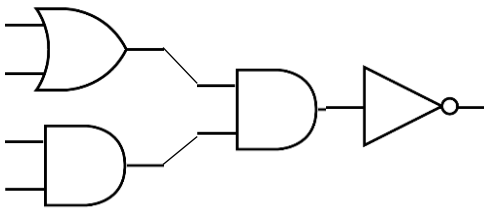
Iff

Cannot verify NO instances!

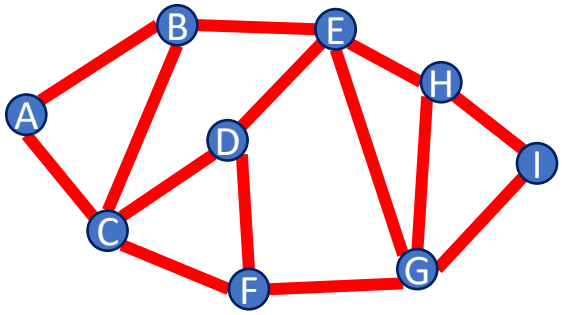
Big idea: Once the instance is in the language, there is a short convincing "witness" / "proof" for this claim.

3-SAT: $(x_1 \vee x_2 \vee \bar{x}_3) \wedge (x_3 \vee x_2 \vee x_4) \wedge (x_1 \vee x_2 \vee x_3)$

CircuitSAT:



LongestPath:



Defining Complexity Class NP

Informal def: decisional problem F is in NP if: whenever a problem YES instance $x \in F$ then this can be proved by providing a **witness** that is **polynomial-time verifiable**.

Defining Complexity Class NP

Formal Definition of NP:

TM or algo.

Definition 15.1 (NP)

We say that $F : \{0,1\}^* \to \{0,1\}$ is in $\mathbf{NP}$ if there exists some integer $a > 0$ and $V : \{0,1\}^* \to \{0,1\}$ such that $V \in \mathbf{P}$ and for every $x \in \{0,1\}^n$,

$$F(x) = 1 \Leftrightarrow \exists_{w \in \{0,1\}^{n^a}} \text{ s.t. } V(xw) = 1 \ . \quad (15.1)$$

The Class P

Functions that can be computed in polynomial time by a standard Turing Machine.

$$\bigcup_{c \in \mathbb{N}} TIME_{TM}(n^c)$$

The Class NP

Functions that can be **verified** in polynomial time by a standard Turing Machine.

Correctness of a **1** output can be *verified* in polynomial time given a witness.

A function $F: \{0,1\}^* \to \{0,1\}$ is in **NP** if there exists some $a \in \mathbb{N}^+$ and $V: \{0,1\}^* \to \{0,1\}$ such that $V \in \mathbf{P}$ and $\forall x \in \{0,1\}^n, F(x) = 1 \Leftrightarrow \exists w \in \{0,1\}^{n^a}$ such that $V(x,w) = 1$.

Example: 3SAT ∈ **NP**

3SAT

Input: A Boolean formula in 3CNF form.

Output: If there is an assignment of values to variables that makes the formula to True, 1. Otherwise, 0.

A function $F: \{0,1\}^* \to \{0,1\}$ is in **NP** if there exists some $a \in \mathbb{N}^+$ and $V: \{0,1\}^* \to \{0,1\}$ such that $V \in \mathbf{P}$ and $\forall x \in \{0,1\}^n, F(x) = 1 \Leftrightarrow \exists w \in \{0,1\}^{n^a}$ such that $V(x,w) = 1$.

Correctness of a **1**-output can be *verified* in polynomial time given a witness.

x: $(x_1 \vee x_2 \vee \bar{x}_3) \wedge (x_3 \vee x_2 \vee x_4) \wedge (x_1 \vee x_2 \vee x_3)$

F: 3SAT

w: $x_1 = 1, x_2 = 1, x_3 = 1, x_4 = 1$

V(x,w): put w into x

Example: 3SAT ∈ **NP**

3SAT

Input: A Boolean formula in 3CNF form.

Output: If there is an assignment of values to variables that makes the formula to True, 1. Otherwise, 0.

A function $F: \{0,1\}^* \to \{0,1\}$ is in **NP** if there exists some $a \in \mathbb{N}^+$ and $V: \{0,1\}^* \to \{0,1\}$ such that $V \in \mathbf{P}$ and $\forall x \in \{0,1\}^n, F(x) = 1 \Leftrightarrow \exists w \in \{0,1\}^{n^a}$ such that $V(x,w) = 1$.

Correctness of a **1**-output can be *verified* in polynomial time given a witness.

F: 3SAT

x: $(x_1 \vee x_2 \vee \bar{x}_3) \wedge (x_3 \vee x_2 \vee x_4) \wedge (x_1 \vee x_2 \vee x_3)$

w: $x_1 = 1, x_2 = 1, x_3 = 1, x_4 = 1$

V(x,w): put w into x

$\Leftrightarrow$, "if and only if":
$F(x) = 0 \Rightarrow$ not exists w such that …

LongestPath ∈ **NP**

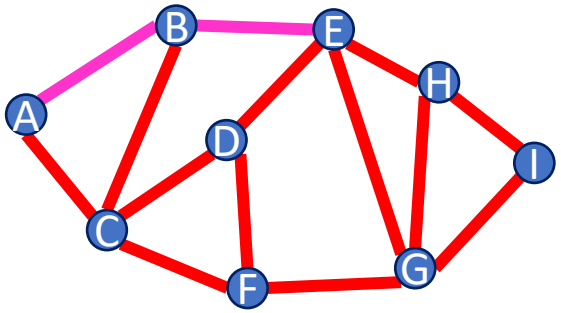
LongestPath

Input: A finite graph $G = (V, E)$, two vertices, $s, t \in V$, and a path length, $\ell \in \mathbb{N}$.

Output: If there is a simple path from s to t in G of length at least ℓ, 1. Otherwise, 0.

A function $F: \{0,1\}^* \to \{0,1\}$ is in **NP** if there exists some $a \in \mathbb{N}^+$ and $V: \{0,1\}^* \to \{0,1\}$ such that $V \in \mathbf{P}$ and $\forall x \in \{0,1\}^n, F(x) = 1 \Leftrightarrow \exists w \in \{0,1\}^{n^a}$ such that $V(x, w) = 1$.

Correctness of a **1**-output can be *verified* in polynomial time given a witness.



F: LongestPath

x: G, s, t, ℓ

w: a path of length ℓ

V: check the path

LongestPath ∈ **NP**

LongestPath-Decision (G, s, t, ℓ)

Input: A finite graph $G = (V, E)$, two vertices, $s, t \in V$, and a path length, $\ell \in \mathbb{N}$.

Output: If there is a simple path from s to t in G of length at least ℓ, 1. Otherwise, 0.

A function $F: \{0,1\}^* \to \{0,1\}$ is in **NP** if there exists some $a \in \mathbb{N}^+$ and $V: \{0,1\}^* \to \{0,1\}$ such that $V \in \mathbf{P}$ and $\forall x \in \{0,1\}^n, F(x) = 1 \Leftrightarrow \exists w \in \{0,1\}^{n^a}$ such that $V(x, w) = 1$.

Correctness of a **1**-output can be *verified* in polynomial time given a witness.

3SAT $\in$ **NP**

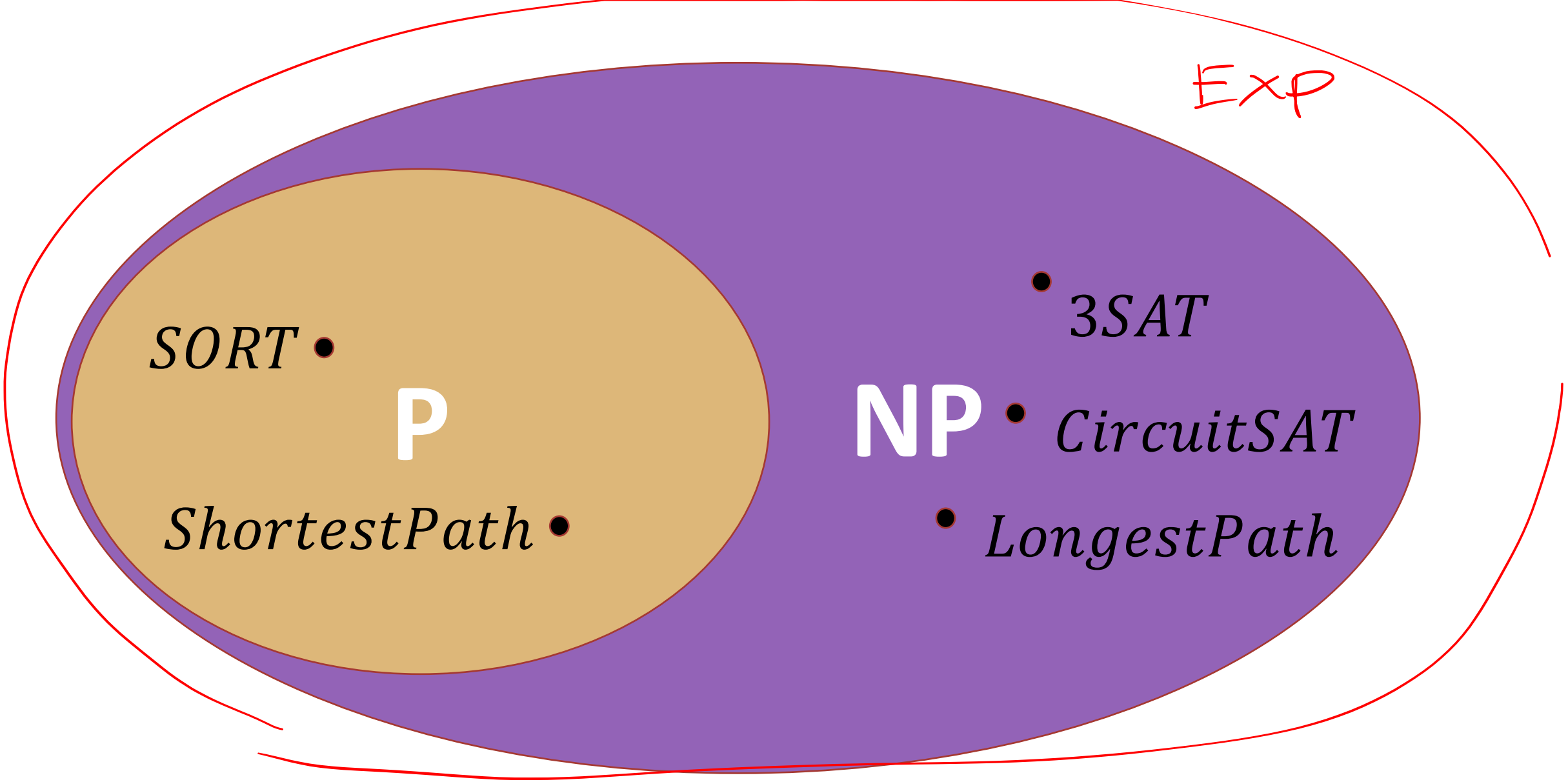
3SAT

Input: A Boolean formula in 3CNF form.

Output: If there is an assignment of values to variables that makes the formula to True, 1. Otherwise, 0.

A function $F: \{0,1\}^* \to \{0,1\}$ is in **NP** if there exists some $a \in \mathbb{N}^+$ and $V: \{0,1\}^* \to \{0,1\}$ such that $V \in \mathbf{P}$ and $\forall x \in \{0,1\}^n, F(x) = 1 \Leftrightarrow \exists w \in \{0,1\}^{n^a}$ such that $V(x,w) = 1$.

Correctness of a **1**-output can be *verified* in polynomial time given a witness.



Unknown if $3SAT \in$ **P**

Known that $3SAT \in$ **NP**

Example: $P \subseteq NP \subseteq EXP$

Class **P** = $\bigcup_{c\in\mathbb{N}} TIME_{TM}(n^c)$

A function $F:\{0,1\}^* \to \{0,1\}$ is in **NP** if there exists some $a \in \mathbb{N}^+$ and $V:\{0,1\}^* \to \{0,1\}$ such that $V \in \mathbf{P}$ and $\forall x \in \{0,1\}^n, F(x) = 1 \Leftrightarrow \exists w \in \{0,1\}^{n^a}$ such that $V(x,w) = 1$.

Suppose $F \in \mathbf{P}$. Can we show that $F \in$ **NP** ?

Yes, proof: $\exists$ M s.t. M(x) = F(x), M is ~~poly-time~~ n^c time

$a =$ 1

$w =$ any string / " " / 1^n

$V(x,w) =$ 1. Output M(x)

Class **P** = $\bigcup_{c\in\mathbb{N}} TIME_{TM}(n^c)$

A function $F: \{0,1\}^* \to \{0,1\}$ is in **NP** if there exists some $a \in \mathbb{N}^+$ and $V: \{0,1\}^* \to \{0,1\}$ such that $V \in \mathbf{P}$ and $\forall x \in \{0,1\}^n, F(x) = 1 \Leftrightarrow \exists w \in \{0,1\}^{n^a}$ such that $V(x,w) = 1$.

Suppose $F \in \mathbf{P}$. Can we show that $F \in$ **NP** ?

Yes, proof:

By $F \in \mathbf{P}$, there is a TM $M, c \in \mathbb{N}$ such that $M(x)$ computes $F(x)$ in n^c steps for all n-bit x.

$a = c$

$w =$ “”

$V(x,w) = M(x)$

We have $F(x) = 1 \leftrightarrow \exists w \in \{0,1\}^{n^a}$ such that $V(x,w) = 1$.

Thus $F \in$ **NP.**

A function $F: \{0,1\}^* \to \{0,1\}$ is in **NP** if there exists some $a \in \mathbb{N}^+$ and $V: \{0,1\}^* \to \{0,1\}$ such that $V \in \mathbf{P}$ and $\forall x \in \{0,1\}^n, F(x) = 1 \Leftrightarrow \exists w \in \{0,1\}^{n^a}$ such that $V(x,w) = 1$.

Class **EXP** = $\bigcup_{c \in \{1,2,3,\dots\}} TIME_{TM}(2^{n^c})$

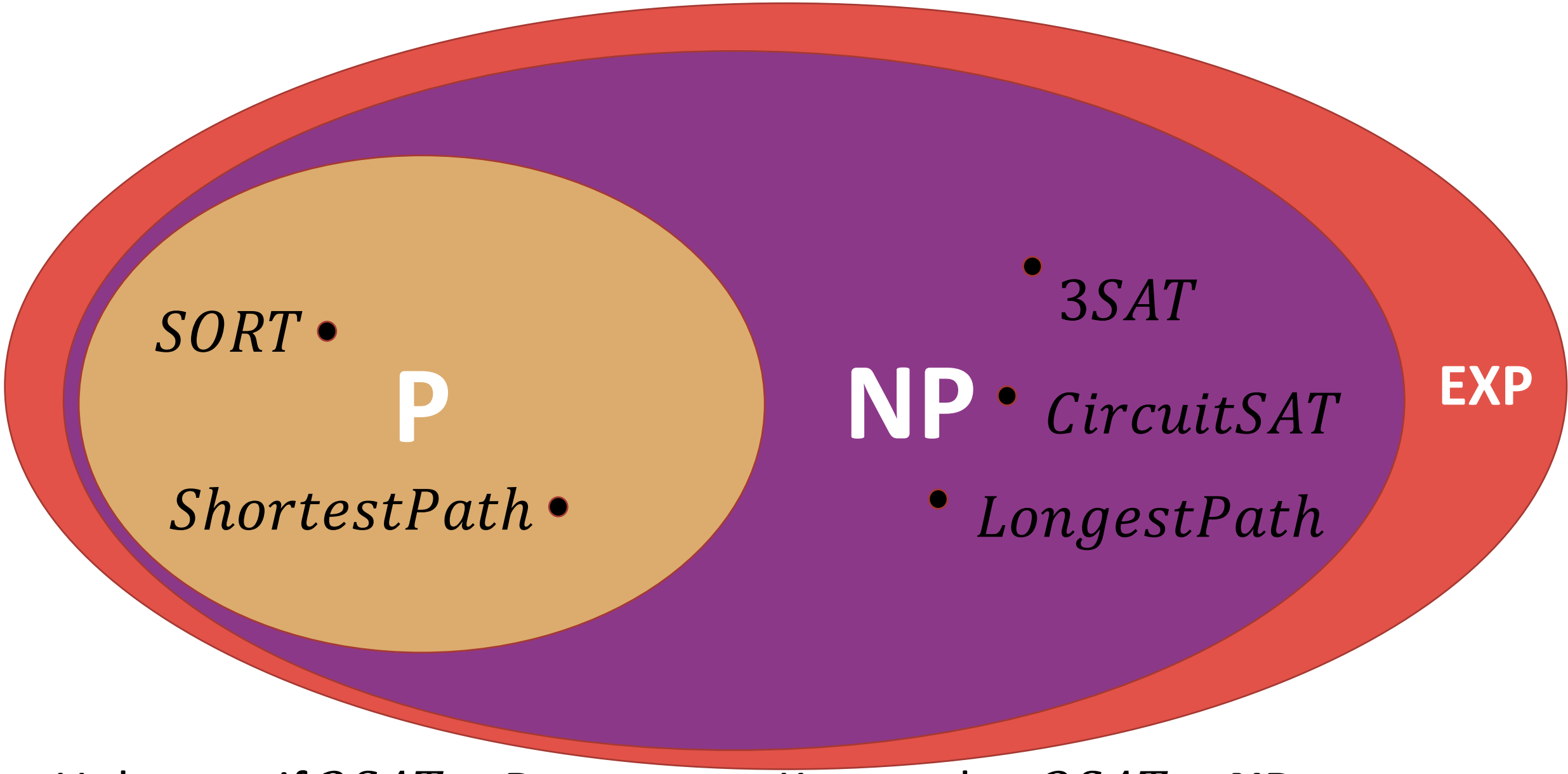
Suppose $F \in \mathbf{NP}$. We have a, V.

Want: $F \in$ **EXP**. TM $M(x)$ computes $F(x)$ in time 2^{n^c}.

$M(x)$:

1. For all $w \in \{0,1\}^{n^a}$
   a. If $V(x, w) = 1$ output 1
2. Output 0

TIME = poly(n) $\cdot 2^{n^a}$



Unknown if $3SAT \in$ **P**

Known that $3SAT \in$ **NP**

More problems. Are they in NP?

Example: PRIME and COMPOSITE

A function $F: \{0,1\}^* \to \{0,1\}$ is in **NP** if there exists some $a \in \mathbb{N}^+$ and $V: \{0,1\}^* \to \{0,1\}$ such that $V \in \mathbf{P}$ and $\forall x \in \{0,1\}^n, F(x) = 1 \Leftrightarrow \exists w \in \{0,1\}^{n^a}$ such that $V(x,w) = 1$.

PRIME(x) =	1	if x is prime
	0	otherwise
COMPOSITE(x) =	1	if x is not prime
	0	otherwise

PRIME = NOT(COMPOSITE)

$\in$ EXP

Yes

Is PRIME in **P**?

Is COMPOSITE in **P**?

?

Eg, COMPOSITE(8633) = 1 bcs $97 \times 89 = 8633$

n-bit

$x = [2^n]$

Is PRIME in **NP**?

Is COMPOSITE in **NP**? YES

F

$y < x$

$V(x, w) = \begin{cases} w < x \\ w \neq 1, w \neq x \\ w \text{ div } x \text{ ?} \end{cases}$

Primality Certificate

Yes! PRIME ∈ NP

Pratt certificates [edit]

The concept of primality certificates was historically introduced by the **Pratt certificate**, conceived in 1975 by Vaughan Pratt,[1] who described its structure and proved it to have polynomial size and to be verifiable in polynomial time. It is based on the Lucas primality test, which is essentially the converse of Fermat's little theorem with an added condition to make it true:

> **Lucas' theorem**: Suppose we have an integer a such that:
>
> - $a^{n-1} \equiv 1 \pmod{n}$,
> - for every prime factor q of $n-1$, it is not the case that $a^{(n-1)/q} \equiv 1 \pmod{n}$.
>
> Then n is prime.

Given such an a (called a *witness*) and the prime factorization of $n-1$, it's simple to verify the above conditions quickly: we only need to do a linear number of modular exponentiations, since every integer has fewer prime factors than bits, and each of these can be done by exponentiation by squaring in $O(\log n)$ multiplications (see big-O notation). Even with grade-school integer multiplication, this is only $O((\log n)^4)$ time; using the multiplication algorithm with best-known asymptotic running time, due to David Harvey and Joris van der Hoeven, we can lower this to $O((\log n)^3(\log \log n))$ time, or using soft-O notation $\tilde{O}((\log n)^3)$.

However, it is possible to trick a verifier into accepting a composite number by giving it a "prime factorization" of $n-1$ that includes composite numbers. For example, suppose we claim that $n = 85$ is prime, supplying $a = 4$ and $n - 1 = 6 \times 14$ as the "prime factorization". Then (using $q = 6$ and $q = 14$):

To be continued ... https://en.wikipedia.org/wiki/Primality_certificate

Annals of Mathematics, **160** (2004), 781–793

PRIMES is in P

By Manindra Agrawal, Neeraj Kayal, and Nitin Saxena*

Abstract

We present an unconditional deterministic polynomial-time algorithm that determines whether an input number is prime or composite.

https://annals.math.princeton.edu/wp-content/uploads/annals-v160-n2-p12.pdf

OPEN!

3UNSAT ∈? **NP**

A function $F: \{0,1\}^* \to \{0,1\}$ is in **NP** if there exists some $a \in \mathbb{N}^+$ and $V: \{0,1\}^* \to \{0,1\}$ such that $V \in \mathbf{P}$ and $\forall x \in \{0,1\}^n, F(x) = 1 \Leftrightarrow \exists w \in \{0,1\}^{n^a}$ such that $V(x,w) = 1$.

3UNSAT

Input: A Boolean formula in 3CNF form.
Output: If there is an assignment of values to variables that makes the formula to True, **0**. Otherwise, **1**.

3UNSAT(x) = NOT(3SAT(x))

Correctness of a **1**-output can be *verified* in polynomial time given a witness.

3SAT

Input: A Boolean formula in 3CNF form.
Output: If there is an assignment of values to variables that makes the formula to True, 1. Otherwise, 0.

NonLongestPath (G, s, t, ℓ)

Input: A finite graph $G = (V, E)$, two vertices, $s, t \in V$, and a path length, $\ell \in \mathbb{N}$.

Output: If there is a simple path from s to t in G of length at least ℓ, 0. Otherwise, 1.

LongestPath (G, s, t, ℓ)

Input: A finite graph $G = (V, E)$, two vertices, $s, t \in V$, and a path length, $\ell \in \mathbb{N}$.

Output: If there is a simple path from s to t in G of length at least ℓ, 1. Otherwise, 0.

NonLongestPath (G, s, t, ℓ) ***= NOT(LongestPath*** (G, s, t, ℓ) ***)***

NonLongestPath $\in$? NP

ShortestPath(G, s, t, ℓ):

1 iff there is a path from s to t in G with $\leq \ell$ steps

NonLongestPath (G, s, t, ℓ)

Input: A finite graph $G = (V, E)$, two vertices, $s, t \in V$, and a path length, $\ell \in \mathbb{N}$.

Output: If there is a simple path from s to t in G of length at least ℓ, 0. Otherwise, 1.

LongestPath (G, s, t, ℓ)

Input: A finite graph $G = (V, E)$, two vertices, $s, t \in V$, and a path length, $\ell \in \mathbb{N}$.

Output: If there is a simple path from s to t in G of length at least ℓ, 1. Otherwise, 0.

NonLongestPath (G, s, t, ℓ) ***= NOT(LongestPath*** (G, s, t, ℓ) ***)***

NonLongestPath(G, A, E, 10) = 1

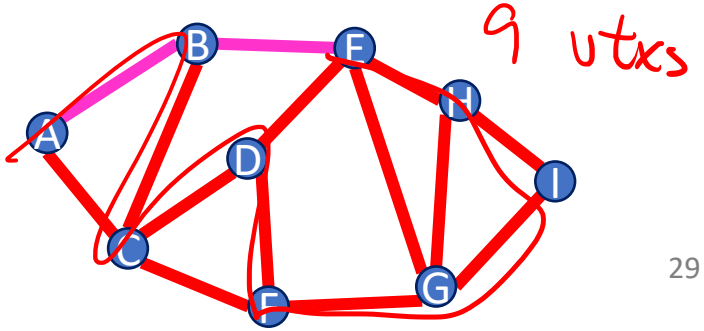
ShortestPath(G, A, E, 10) = 1

NonLongestPath(G, A, E, 5) = 0

ShortestPath(G, A, E, 5) = 1

ShortestPath(G, s, t, ℓ):

1 iff there is a path from s to t in G with $\leq \ell$ steps



Class **co-NP**

A function $F: \{0,1\}^* \to \{0,1\}$ is in **NP** if there exists some $a \in \mathbb{N}^+$ and $V: \{0,1\}^* \to \{0,1\}$ such that $V \in \mathbf{P}$ and $\forall x \in \{0,1\}^n, F(x) = 1 \Leftrightarrow \exists w \in \{0,1\}^{n^a}$ such that $V(x,w) = 1$.

Class **co-NP** = $\{F: F$ is a Boolean function and $NOT\big(F(x)\big) \in$ **NP** $\}$

3UNSAT

Input: A Boolean formula in 3CNF form.

Output: If there is an assignment of values to variables that makes the formula to True, **0**. Otherwise, **1**.

3SAT

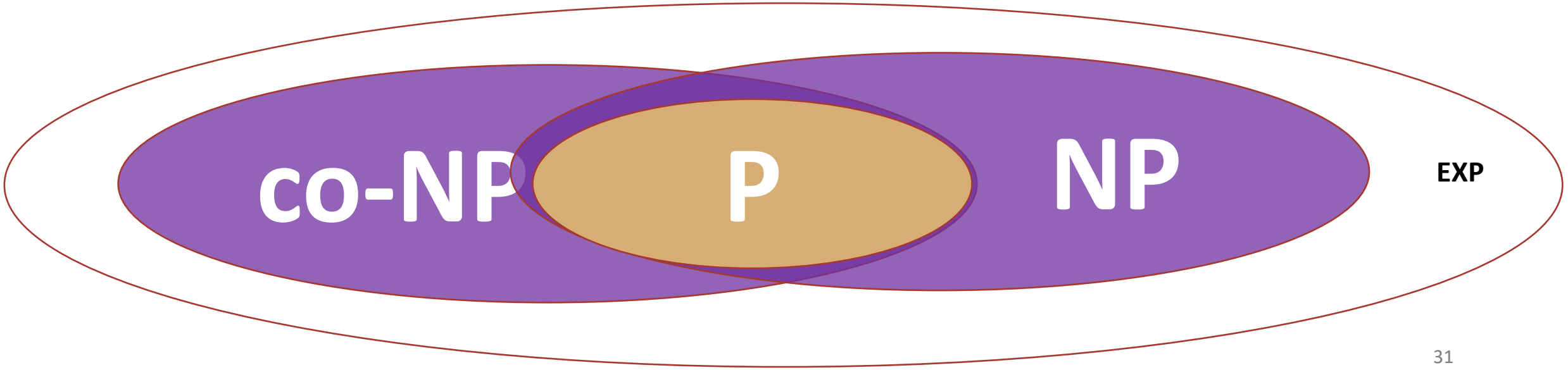
Input: A Boolean formula in 3CNF form.

Output: If there is an assignment of values to variables that makes the formula to True, 1. Otherwise, 0.

Class **co-NP**

A function $F: \{0,1\}^* \to \{0,1\}$ is in **NP** if there exists some $a \in \mathbb{N}^+$ and $V: \{0,1\}^* \to \{0,1\}$ such that $V \in \mathbf{P}$ and $\forall x \in \{0,1\}^n, F(x) = 1 \Leftrightarrow \exists w \in \{0,1\}^{n^a}$ such that $V(x,w) = 1$.

Class **co-NP** = $\{F: F$ is a Boolean function and $NOT\big(F(x)\big) \in$ **NP** $\}$



Open Problem: P = NP?
NP-Hard and NP-Complete

Millennium Problem ($1 million prize)

Clay Mathematics Institute

About Programs & Awards People The Millennium Prize Problems Online resources Events News

Home — Millennium Problems — P vs NP

Unsolved

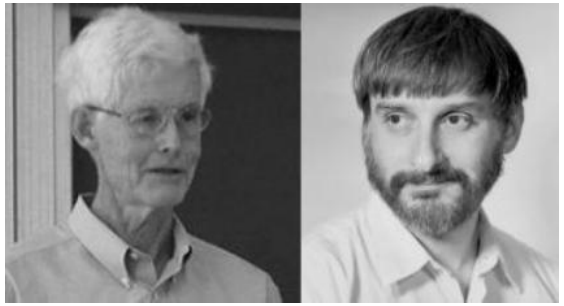
P vs NP

If it is easy to check that a solution to a problem is correct, is it also easy to solve the problem? This is the essence of the P vs NP question. Typical of the NP problems is that of the Hamiltonian Path Problem: given N cities to visit, how can one do this without visiting a city twice? If you give me a solution, I can easily check that it is correct. But I cannot so easily find a solution.

https://www.claymath.org/millennium/p-vs-np/

Yet another problem in NP

- How to accommodate 400 students in a dorm?
- Space is limited: only 100 places
- Some pairs of incompatible students such that no pair appear in final choice



Stephen Cook and Leonid Levin

https://www.claymath.org/millennium/p-vs-np/

Open Problem: $\mathbf{P} = \mathbf{NP}$?

Is it harder to find a solution to a problem or to check if a provided solution is correct?

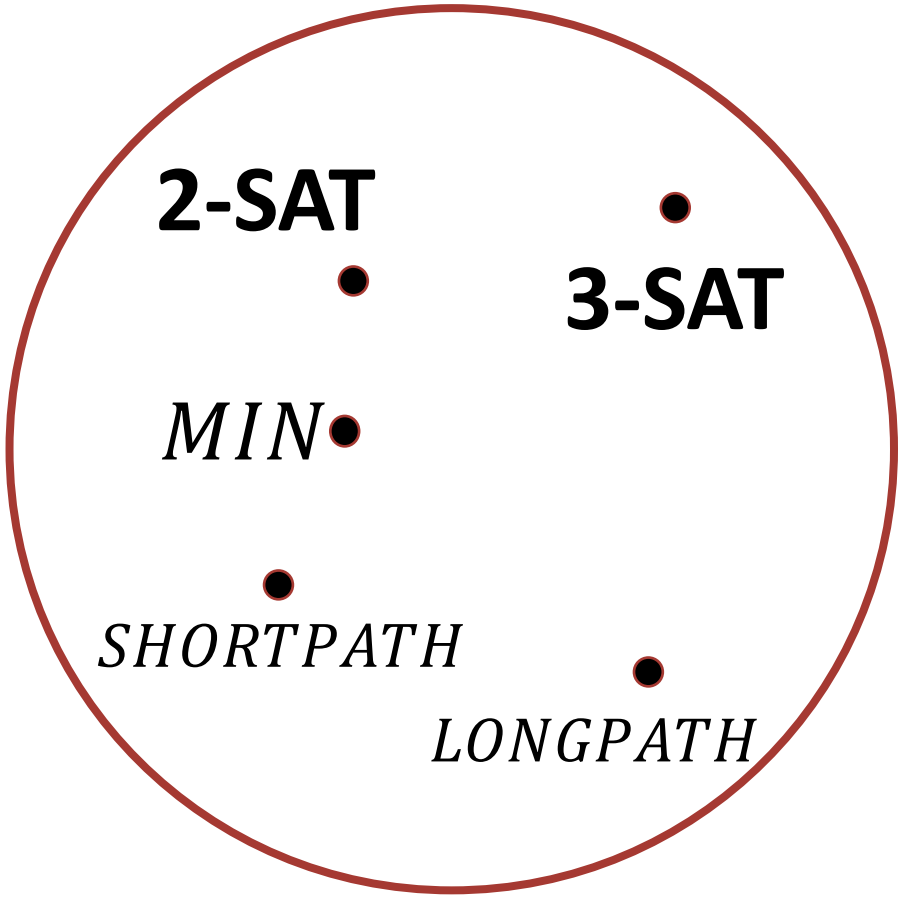
Yes: $\mathbf{P} \subsetneq \mathbf{NP}$

There are some problems where it is hard to find a solution, but if given an answer it is easy to check it.

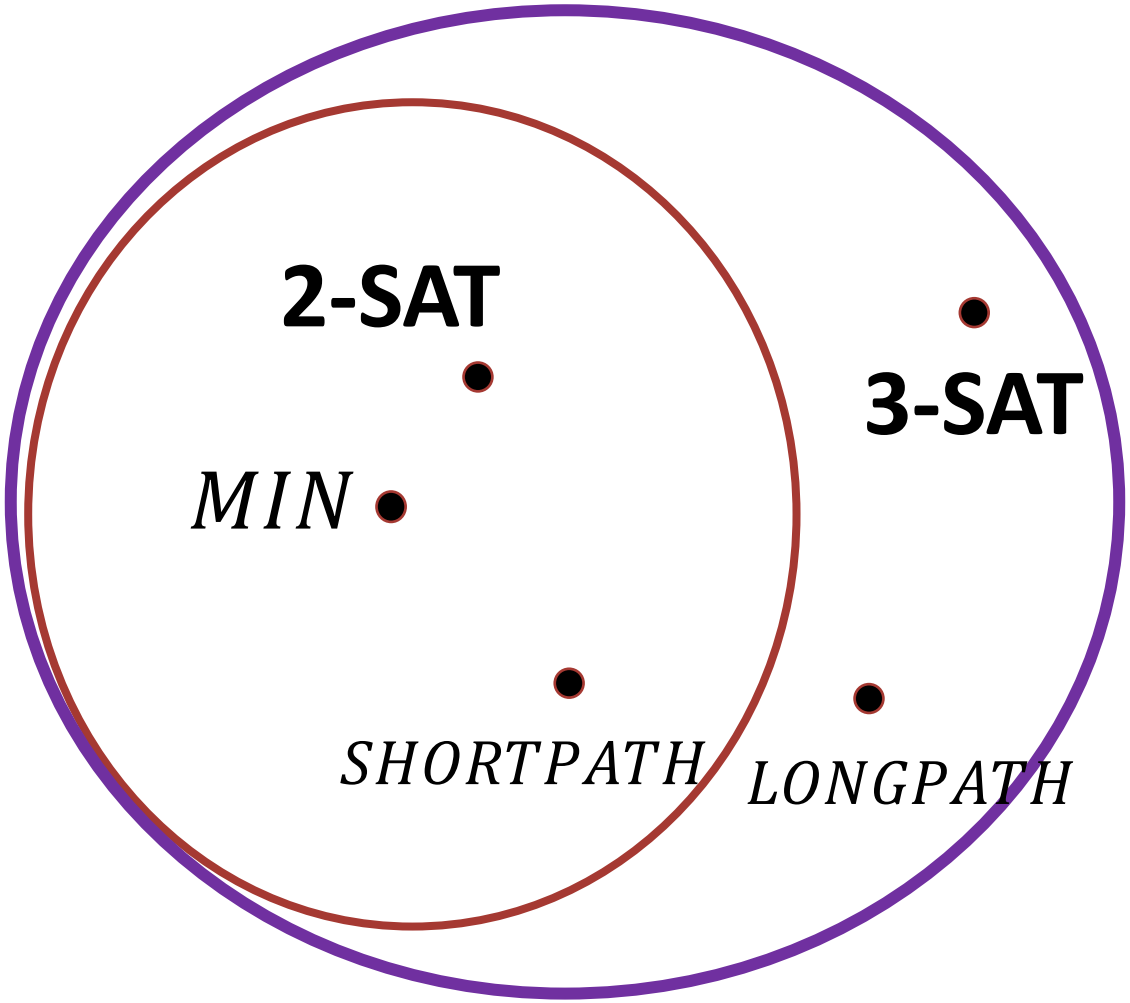
No: $\mathbf{P} = \mathbf{NP}$

If it is easy to check if a solution to a problem is correct, it is also easy to find a solution to that problem.

The answer to this question is unknown! This is the most famous open problem in mathematics and computer science.

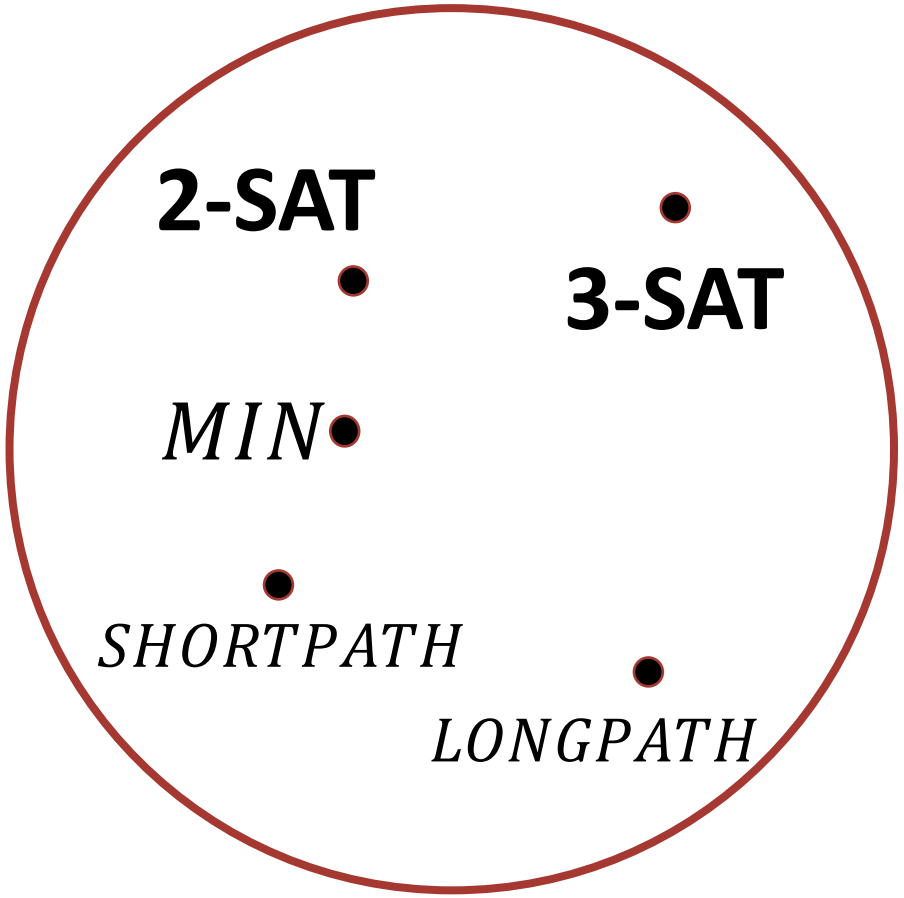


If **P = NP**

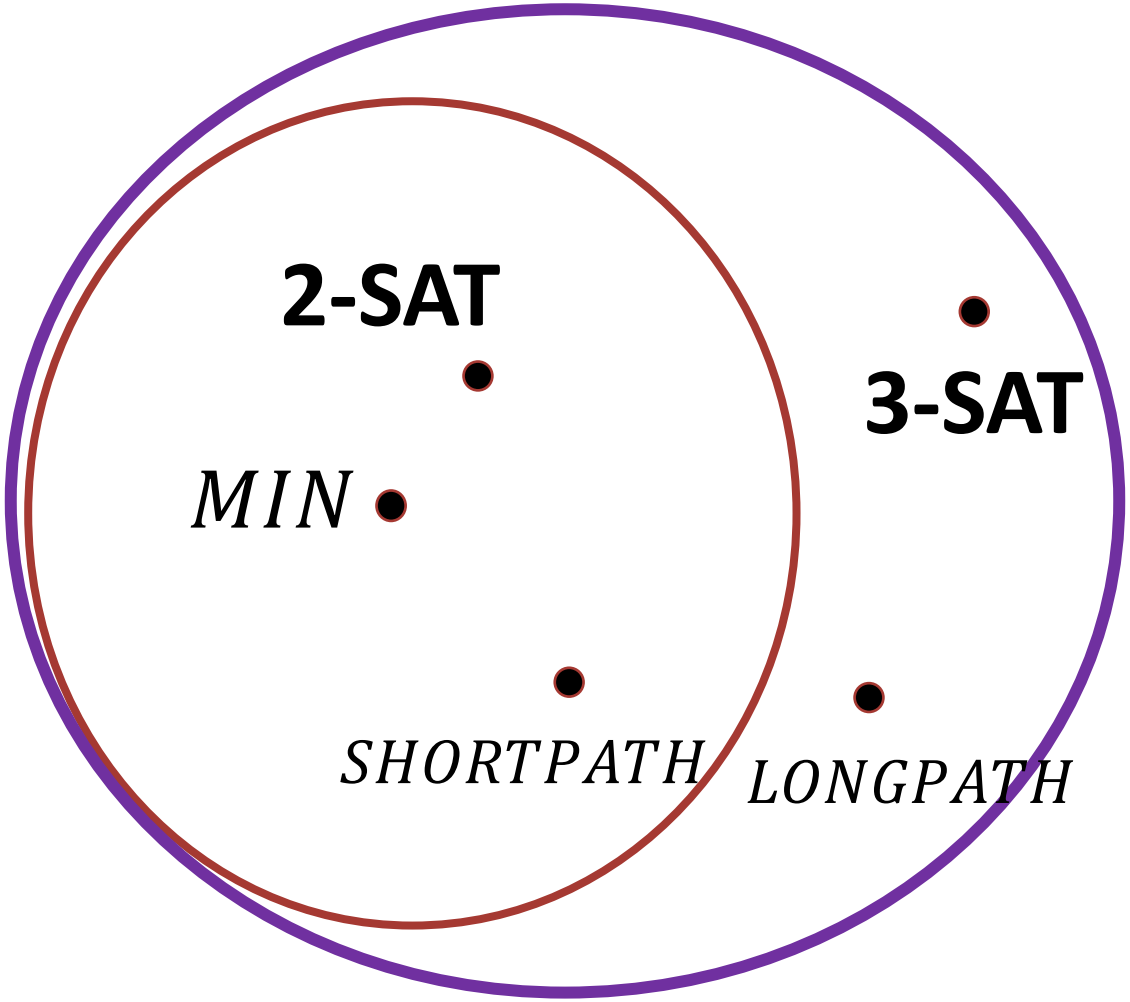


If **P** $\subsetneq$ **NP**

*How could we **prove** P = NP?*

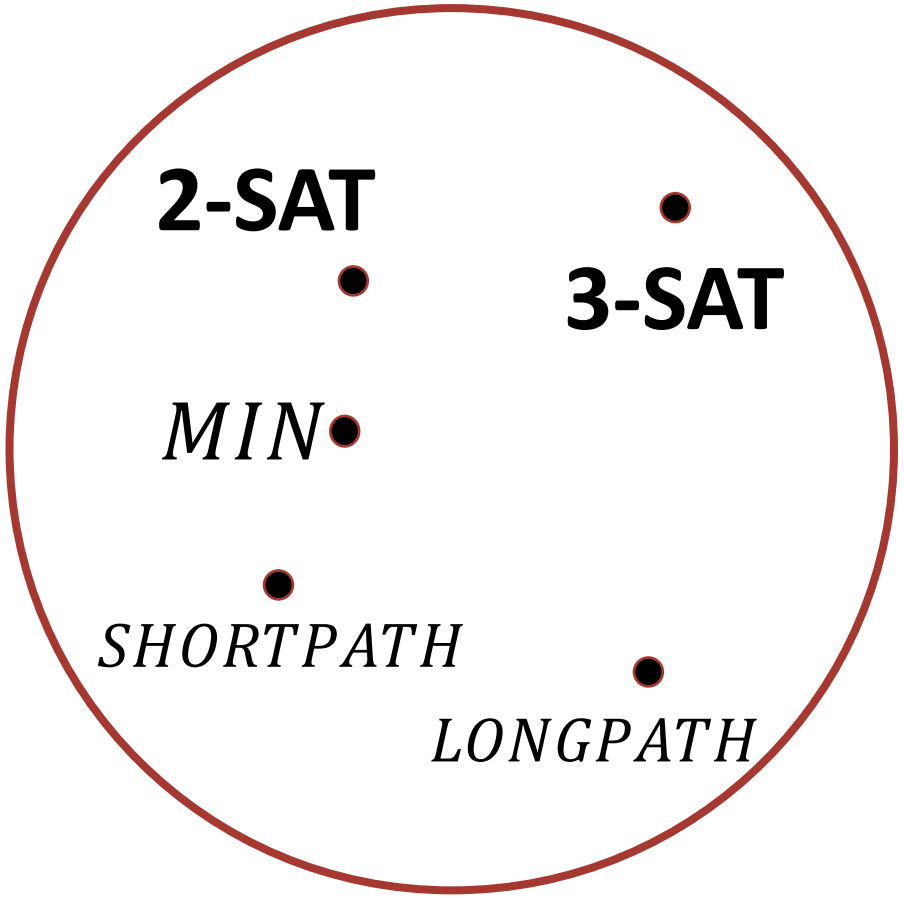


If **P = NP**

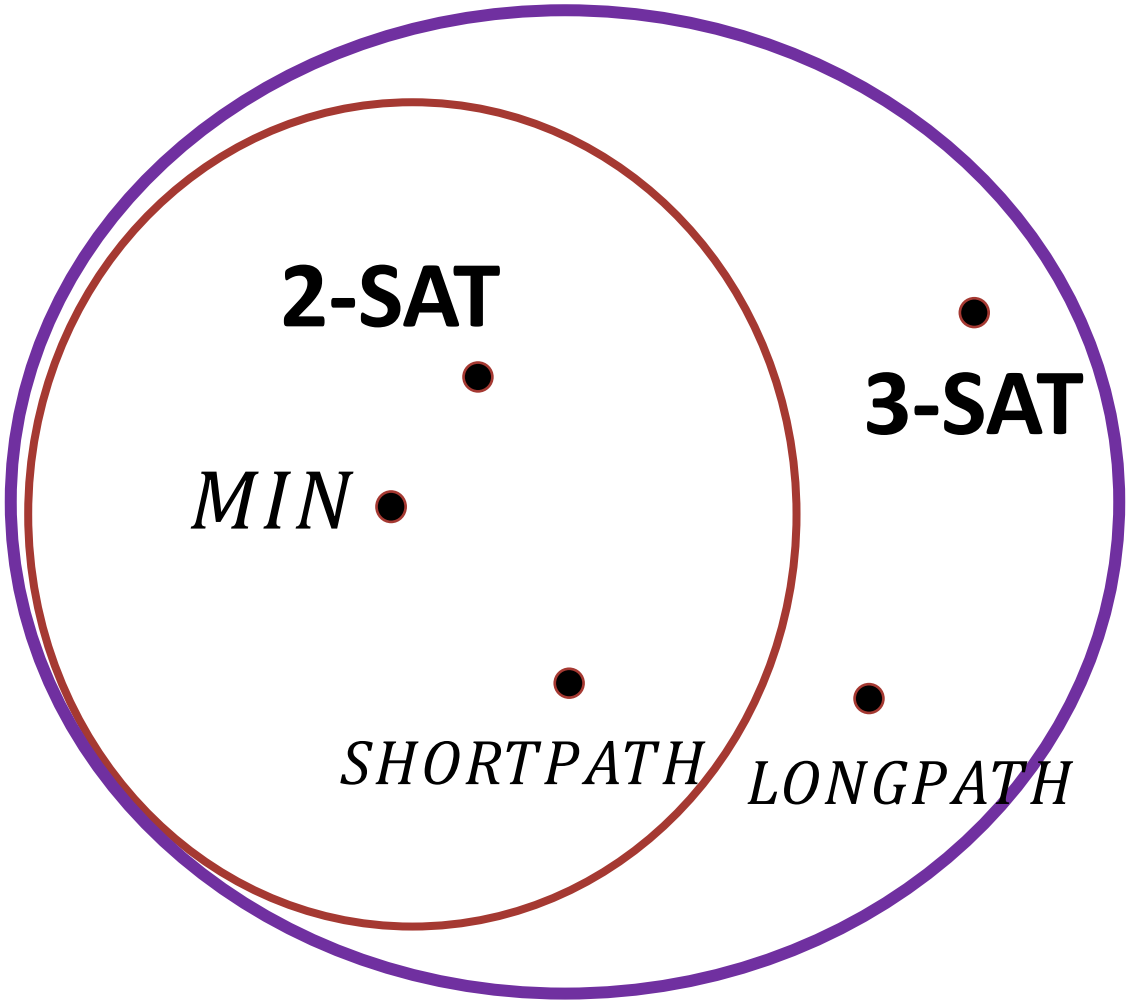


If **P $\subsetneq$ NP**

*How could we **prove** P ⊊ NP?*



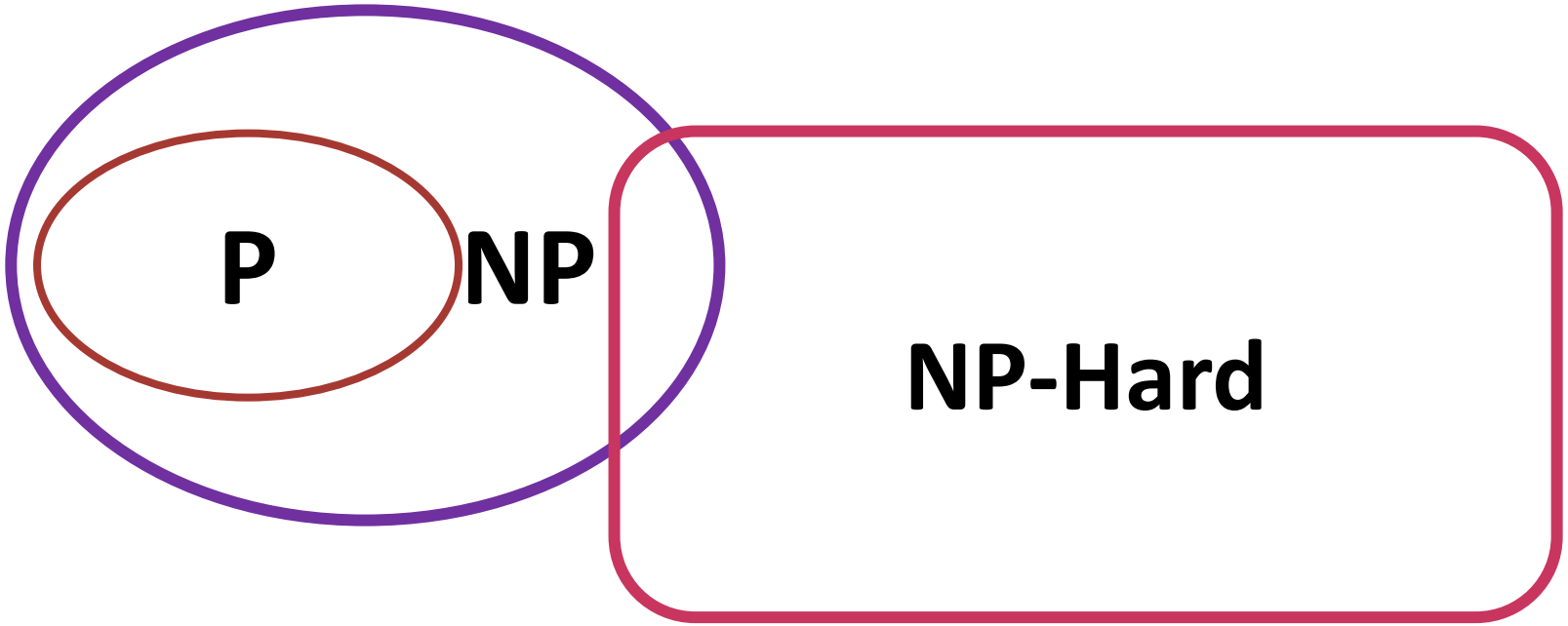
If **P = NP**



If **P ⊊ NP**

Complexity Class: NP-Hard

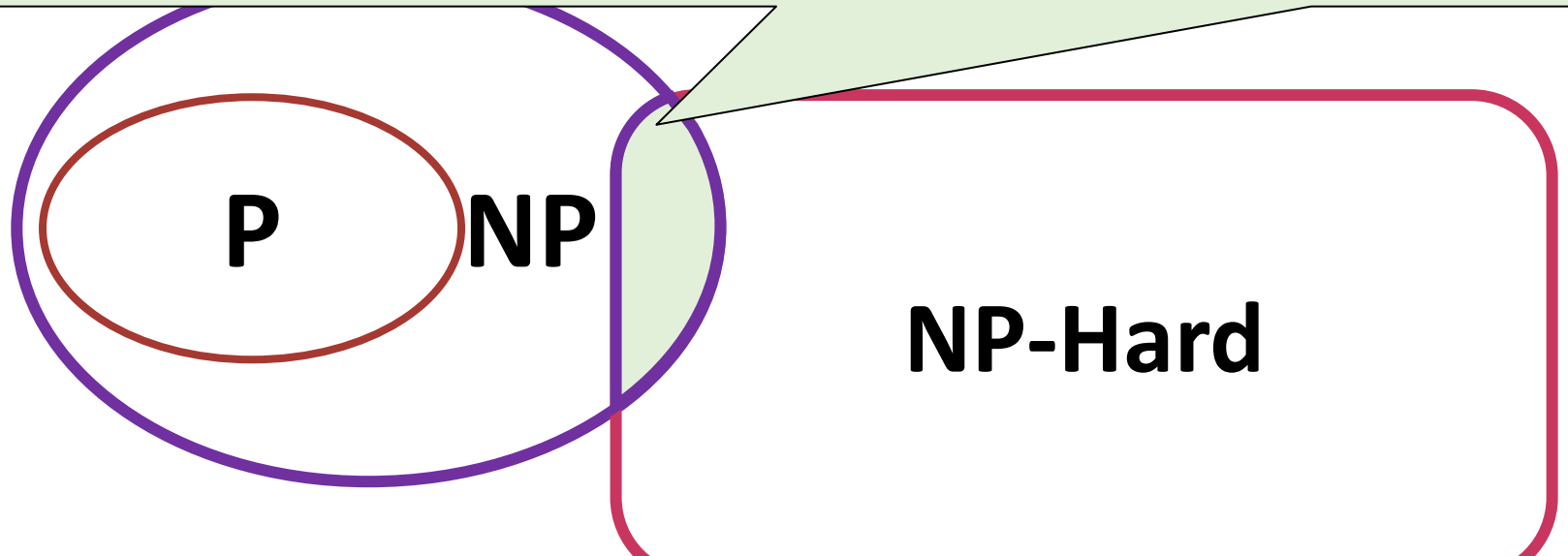
Definition: A Boolean function G is **NP-Hard** if every $F \in \mathbf{NP}$ can be reduced to G: $F \leq_p G$.



NP-Complete

Definition: A Boolean function G is **NP-Hard** if every $F \in \mathbf{NP}$ can be reduced to G: $F \leq_p G$.

Definition: A function G is **NP-Complete** if $G \in \mathbf{NP}$ and G is **NP-Hard**.



Cook-Levin Theorem

Cook-Levin Theorem

(Theorem 15.6 in the TCS Book):

For every $F \in \mathbf{NP}$, $F \leq_p$ **3SAT.**

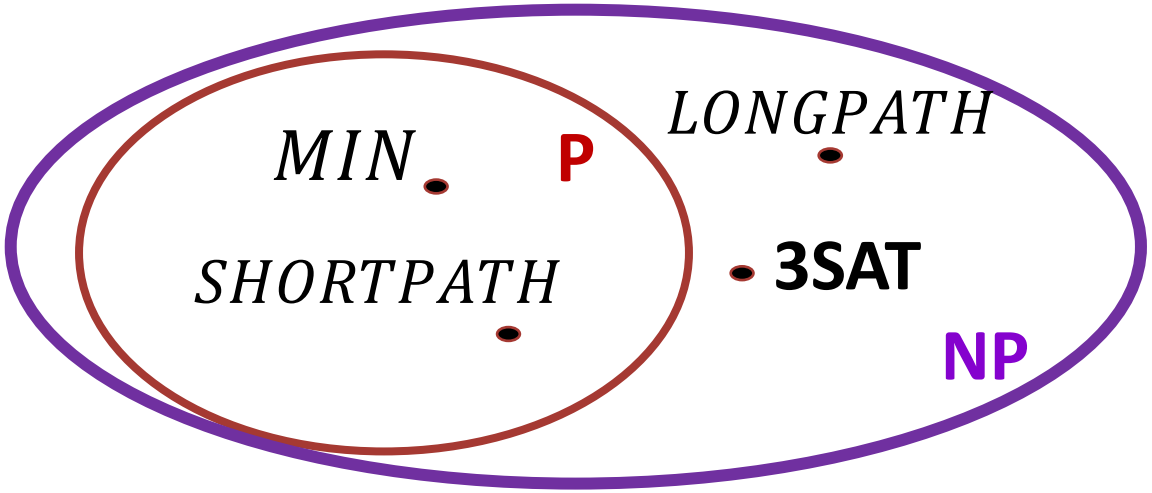
Definition: A Boolean function G is **NP-Hard** if every $F \in \mathbf{NP}$ can be reduced to G: $F \leq_p G$.

Equivalently: **3SAT** is (in) **NP-Hard**

Making Progress on $\mathbf{P} \subsetneq \mathbf{NP}$

Cook-Levin Theorem: For every $F \in \mathbf{NP}$, $F \leq_p$ **3SAT.**

If **3SAT** $\in$ **P**



If **3SAT** $\notin$ **P**

Logistics

HW 8 due this Friday.
HW 9 coming soon.

No Quiz this week.

Plan

Proof of Cook-Levin Theorem

Textbook [TCS] Section 15.3-15.4

https://introtcs.org/public/lec_13_Cook_Levin.html#the-nandsat-problem-and-why-it-is-mathbfnp-hard

NP, NP Completeness